ARQUITETURA DE ALTA PERFORMANCE: MYSQL & ELASTICSEARCH

APIs de Consulta em Tempo Real com Sincronização CDC para Tabelas Massivas

RESPONSÁVEL TÉCNICO:
Pedro Moacyr Barcelos Neto (M.Sc. UFMS)

OBJETIVO DO PROJETO:
Espelhamento CDC & Consultas Sub-milissegundo

DATA / VERSÃO:
Outubro de 2026 | v1.0 Executiva

1. DIAGNÓSTICO DO CENÁRIO & JUSTIFICATIVA ARQUITETURAL

Em bancos de dados relacionais com tabelas muito volumosas e alta taxa de concorrência entre operações de escrita (INSERT, UPDATE, DELETE) e consultas simultâneas, executar buscas diretamente no MySQL resulta em severa contenção de bloqueios (Table e Row Locks), degradação de I/O em disco e consumo de até 90% da CPU. A tentativa de resolver esse cenário apenas com índices tradicionais do MySQL se torna ineficiente quando há necessidade de busca textual flexível, filtros multifatoriais e paginações extensas.

A solução de engenharia consolidada na indústria consiste no desacoplamento completo da carga de leitura através do padrão Dual-Source / CQRS: o MySQL permanece estritamente como a Fonte Única da Verdade (Single Source of Truth) para manter integridade transacional ACID, enquanto um cluster Elasticsearch atua como espelho analítico otimizado, atendendo 100% das consultas complexas em menos de 15 milissegundos.

2. ARQUITETURA DA SOLUÇÃO (CHANGE DATA CAPTURE - CDC)



- **Escritas e Consistência ACID:** Todas as mutações ocorrem prioritariamente no MySQL, preservando integridade referencial, travas transacionais e chaves.
- **Streaming Contínuo via CDC:** Captura instantânea de alterações via Binary Log / Outbox pattern, sem necessidade de consultas periódicas pesadas no bar
- **Consistência Estrita em Deleções:** Registros excluídos no MySQL emitem evento CDC imediato que purga o documento no Elastic, eliminando dados fant
- **Consultas Otimizadas na API:** Endpoints REST assíncronos (FastAPI) com paginação eficiente (Search After), filtros combinados e busca fonética/fuzzy.

3. PROTÓTIPOS E CÓDIGO-FONTE HOMOLOGADOS (LINKS PÚBLICOS)

REPOSITÓRIO COMPLETO NO GITHUB (FASTAPI + DOCKER COMPOSE + MOTOR CDC):
`https://github.com/barcelosneto/mysql-elasticsearch-sync-api`

Inclui código-fonte da API, schemas SQLAlchemy/Pydantic, scripts de inicialização MySQL e testes automatizados.

DEMONSTRAÇÃO INTERATIVA AO VIVO (SIMULADOR DE MUTAÇÕES E BUSCAS):
`https://barcelosneto.github.io/site-portfolio/demos/mysql-elastic-api/`

Permite disparar operações de INSERT, UPDATE e DELETE no MySQL e acompanhar a sincronização CDC instantânea no Elastic.

4. BENCHMARK DE PERFORMANCE COMPARATIVO (5.000.000 DE REGISTROS)

Métrica de Avaliação	MySQL 8.0 (Direto)	Elasticsearch 8.13 (Mirror)	Ganho / Eficiência
Busca Textual Parcial (LIKE %termo%)	1.840 ms	12 ms	**153x mais rápido**
Filtro Multifatorial com Paginação	680 ms	8 ms	**85x mais rápido**
Carga de CPU no Banco durante Leituras	85% a 95% de uso	0.0% (Isolado)	**Zero lock contention**
Latência de Replicação CDC (Lag)	Não aplicável	Menor que 100 ms	**Sincronização em tempo real**
Consistência em Exclusões (DELETE)	Nativo no MySQL	Purga imediata no índice	**Sem registros fantasmas**

5. PLANO DE EXECUÇÃO & METODOLOGIA DE IMPLEMENTAÇÃO

01 **Fase 1: Mapeamento de Schemas & Modelagem no Elasticsearch**
Análise estrutural das tabelas MySQL de alta volumetria, definição de tipos de dados, chaves primárias e mapeamento de índice invertido com analyzers adequados para buscas parciais.

02 **Fase 2: Implantação do Pipeline de CDC (Change Data Capture)**
Configuração do leitor de Binary Log com envio em lote assíncrono para o Elasticsearch. Tratamento atômico de mutações: criação, atualização e purga imediata de registros deletados.

03 **Fase 3: Desenvolvimento dos Endpoints REST de Consulta & Gestão**
Construção de APIs de alta velocidade (FastAPI / Python ou Node.js / C#), implementando filtros combinados, busca aproximada (Fuzzy), ordenação dinâmica e paginação leve.

04 **Fase 4: Containerização (Docker Compose) e Scripts de Carga Inicial**
Entrega de orquestração completa em contêineres com MySQL, cluster Elasticsearch, Kibana e workers de replicação com auto-restart e scripts para migração da base inicial existente.

05 **Fase 5: Testes de Carga, Validação de Consistência e Homologação**
Execução de testes de estresse sob cenários de escrita intensa, auditoria de latência de replicação (zero desync) e entrega da documentação interativa Swagger/OpenAPI.

6. PERFIL DO PROFISSIONAL RESPONSÁVEL

Pedro Moacyr Barcelos Neto
Mestre em Computação Aplicada — Universidade Federal de Mato Grosso do Sul (UFMS)
Especialista em Engenharia de Software, Backend de Alta Performance, Microsserviços e Bancos de Dados em Larga Escala.
E-mail: pedrobarcelos1980@gmail.com | LinkedIn: linkedin.com/in/barcelosneto | GitHub: github.com/barcelosneto